



SUPERVISED LEARNING ALGORITHM

Logistic Regression, Support Vector Machines, Decision Trees, and KNN

A tour of classifiers with scikit-learn: probabilistic models, maximum-margin classification, kernel methods, and instance-based learning.

Dr. Md. Shahidur Rahman
Professor, DoCSE, SUST
rahmanms@sust.edu

ROADMAP

What We'll Cover

01 Choosing a Classifier

The supervised-learning workflow and the no-free-lunch theorem

02 Logistic Regression

Modeling class probabilities with the sigmoid and log-likelihood

03 Overfitting & Regularization

Bias-variance trade-off and L2 shrinkage

04 Support Vector Machines

Maximum-margin classification and soft margins

05 Decision Trees & Random Forests

Splitting on the most informative features, combining trees into a robust ensemble

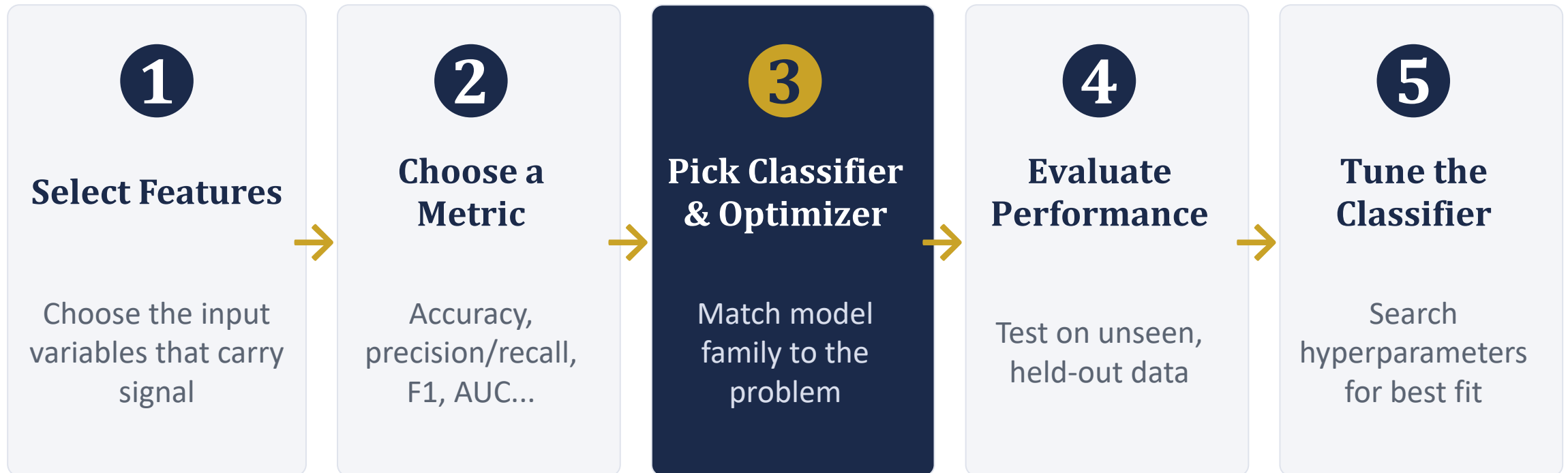
06 K-Nearest Neighbors

Instance-based, non-parametric classification

FOUNDATIONS

Choosing a Classification Algorithm

No single classifier wins in every scenario — the *"no free lunch" theorem*. The right model always depends on the specifics of the problem.



SECTION 02

Logistic Regression

A probabilistic classifier that models the likelihood of class membership

MOTIVATION

From the Perceptron to a Probabilistic Model

- When classes are not linearly separable, perceptron weights never stop updating — at least one-point stays misclassified every epoch.
- Logistic regression is a far better default: it converges even on non- linearly separable data.
- Despite the name, logistic regression is a classification model, not a regression model.

Perceptron

Hard step function → only converges if classes are perfectly linearly separable.

Logistic Regression

Smooth sigmoid activation → always converges, and outputs a class probability.

THE LOGISTIC MODEL

Odds Ratio & the Logit Function

- Logistic regression is the 'go to' model for binary classification, and extends naturally to multiclass problems.
- The odds ratio compares the probability of the positive class to the probability of the negative class.
- The logit function is the log of the odds ratio — it maps probabilities (0,1) onto the full real line.

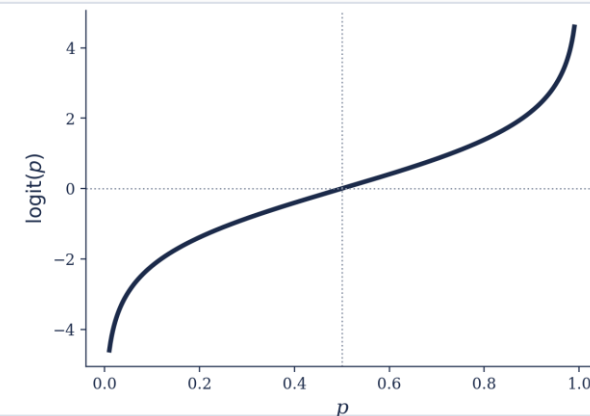
$$\frac{p}{1-p}$$

odds ratio

$$\text{logit}(p) = \log \frac{p}{1-p}$$

logit function

p = probability of the positive class (y = 1) — e.g. the probability a patient has a given disease.



The logit is the inverse of the logistic (sigmoid) function: reversing $f(x) = y$ means $g(y) = x$.

Modeling the Logit Function

- We model the logit as a linear combination of features — the dot product of feature values and weights.
- $p(y=1 | \mathbf{x})$ is the conditional probability that a sample with features $\mathbf{x}$ belongs to class 1.
- Solving for p gives the logistic (sigmoid) form used to make predictions.

$$\text{logit}(p(y = 1 | \mathbf{x})) = \sum_{i=0}^m w_i x_i = \mathbf{w}^T \mathbf{x}$$

linear combination of weights and features

$$p(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

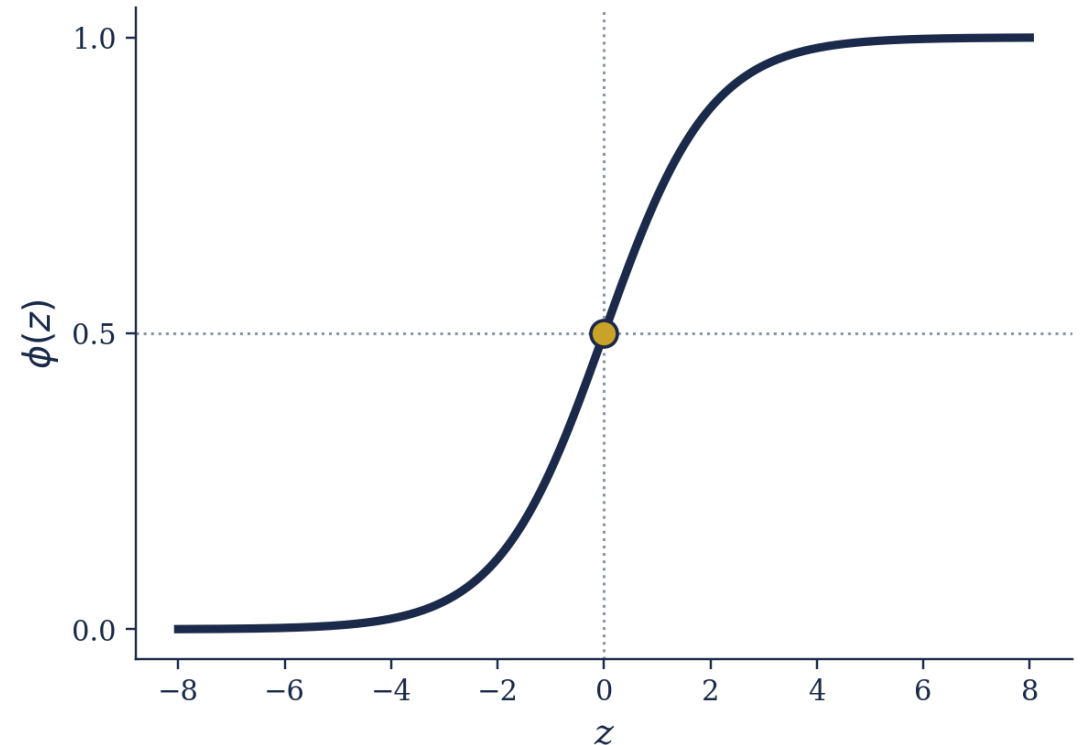
equivalent probability form

THE LOGISTIC MODEL

The Logistic Sigmoid Function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

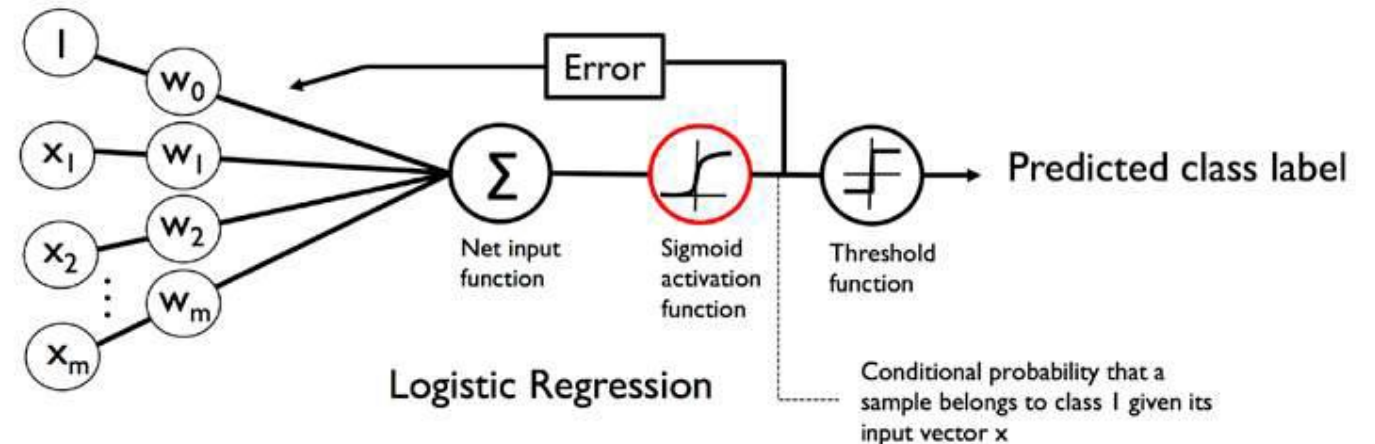
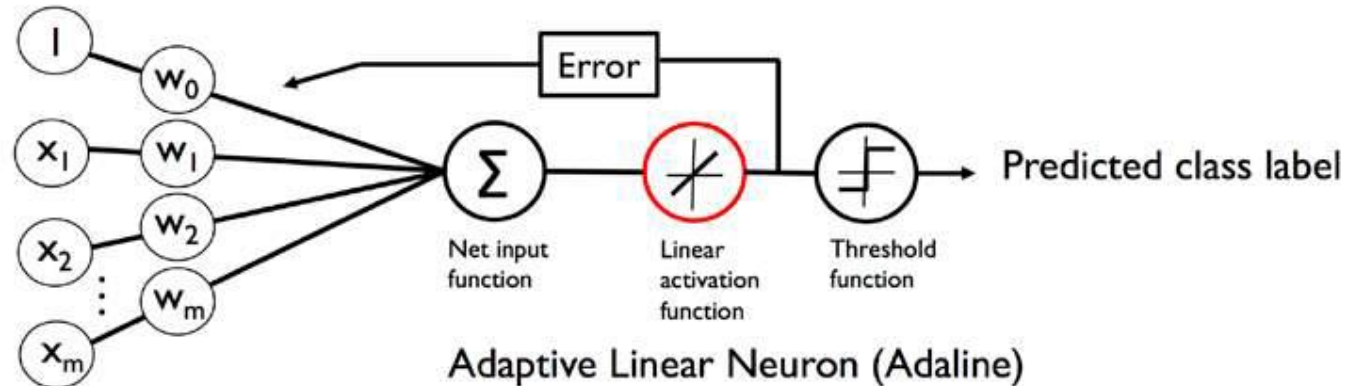
- Also known simply as the sigmoid function.
- Produces the characteristic S-shaped curve.
- Squashes any real-valued input z into the range $(0, 1)$ — **ideal for representing a probability.**
- **Weather prediction with probability**



THE LOGISTIC MODEL

Relationship with Adaline

- Adaline uses the **identity function as its activation** — the net input passes straight through.
- Logistic regression **swaps in the sigmoid function** as the activation, converting the net input into a probability before the final class decision.



From Probability to a Binary Decision

- The sigmoid's output is directly interpreted as a class probability —
e.g. $P(y=1 | \mathbf{x}; \mathbf{w}) = 0.8$.
- A quantizer converts that probability into a binary outcome using a 0.5 threshold.
- Equivalently, the decision only depends on the sign of the net input z .
- Many real applications — such as weather forecasting — care about the probability itself, not just the hard label.

$$\hat{y} = 1 \text{ if } \phi(z) \geq 0.5, \text{ else } \hat{y} = 0$$

$$p(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

Defining the Likelihood

- Previously (Adaline), we minimized the sum-squared-error cost function.
- For logistic regression we instead derive a cost function from the likelihood of observing the labels we actually saw.
- The **likelihood $L(\mathbf{w})$ is the probability** of the full label vector $\mathbf{y}$ given the data $\mathbf{x}$ and weights $\mathbf{w}$.

$$L(\mathbf{w}) = \prod_{i=1}^n (\phi(z^{(i)}))^{y^{(i)}} (1 - \phi(z^{(i)}))^{1 - y^{(i)}}$$

Product over all n samples of each sample's probability of its observed label.

The Log-Likelihood Function

- In practice, it's much easier to work with the natural log of the likelihood.
- The log turns a product of many small numbers into a sum — easier to differentiate and far less prone to numerical underflow.

$$l(\mathbf{w}) = \sum_{i=1}^n [y^{(i)} \log \phi(z^{(i)}) + (1 - y^{(i)}) \log (1 - \phi(z^{(i)}))]$$

Easier derivative

Fewer underflow issues

The Cost Function

- Flipping the sign of the log-likelihood turns **maximization into a minimization** problem — the standard convention for a cost function.
- This cost function can now be minimized with gradient descent, exactly as in the Adaline chapter.

$$J(\mathbf{w}) = \sum_{i=1}^n \left[-y^{(i)} \log \phi(z^{(i)}) - (1 - y^{(i)}) \log (1 - \phi(z^{(i)})) \right]$$

Deriving the Gradient

- Taking the partial derivative of the log-likelihood with respect to weight w_j :

$$\frac{\partial}{\partial w_j} l(\mathbf{w}) = \left(y \frac{1}{\phi(z)} - (1 - y) \frac{1}{1 - \phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z)$$

- This requires the derivative of the sigmoid, which can be expressed in terms of $\phi(z)$.

$$\frac{\partial}{\partial z} \phi(z) = \frac{\partial}{\partial z} \frac{1}{1 + e^{-z}} = \frac{1}{(1 + e^{-z})^2} e^{-z} = \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}} \right) = \phi(z)(1 - \phi(z))$$

- Substituting this back in and simplifying collapses the whole expression to

$$\frac{\partial}{\partial w_j} l(\mathbf{w}) = \left(y^{(i)} - \phi(z^{(i)}) \right) x_j^{(i)}$$

$$\begin{aligned} \left(y \frac{1}{\phi(z)} - (1 - y) \frac{1}{1 - \phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z) &= \left(y \frac{1}{\phi(z)} - (1 - y) \frac{1}{1 - \phi(z)} \right) \phi(z)(1 - \phi(z)) \frac{\partial}{\partial w_j} z \\ &= \left(y(1 - \phi(z)) - (1 - y)\phi(z) \right) x_j \\ &= (y - \phi(z)) x_j \end{aligned}$$

Deriving the Gradient

- To find the weights that maximize the log-likelihood we perform the update for each weight as follows:

$$w_j := w_j + \eta \sum_{i=1}^n \left(y^{(i)} - \phi(z^{(i)}) \right) x_j^{(i)}$$

- Maximizing the log-likelihood is equal to minimizing the cost function J , we can write the gradient descent update rule as below, which is similar to gradient descent rule for Adaline

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_{i=1}^n \left(y^{(i)} - \phi(z^{(i)}) \right) x_j^{(i)}$$

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, \quad \Delta \mathbf{w} = -\eta \nabla J(\mathbf{w})$$

SECTION 03

Overfitting & Regularization

Balancing model complexity against generalization to unseen data

Overfitting & Underfitting

High variance

Overfitting

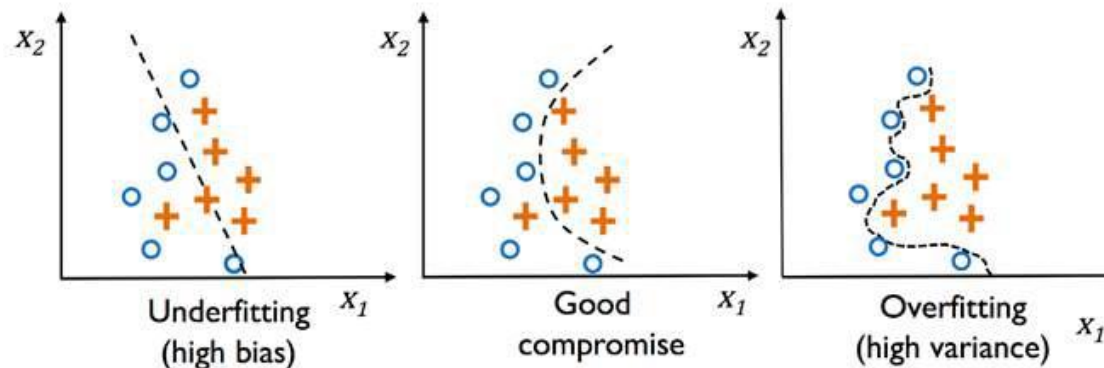
Model fits training data well but fails to generalize to unseen (test) data. Usually caused by a model that is too complex.

High bias

Underfitting

Model isn't complex enough to capture the underlying pattern — it performs poorly even on training data.

Both symptoms share the same consequence: low performance on unseen data.



REGULARIZATION

Regularization: Taming Model Complexity

Regularization: adding information to solve an ill-posed problem or prevent overfitting. An ill-posed problem is one where the solution doesn't exist or isn't unique.

- Regularization tunes the complexity of the model.
- It filters out noise in the training data.
- As a result, it prevents overfitting while preserving genuine signal.

Underfitting (high bias)

*Too simple to
capture the pattern*

Good compromise

*Complexity matched
to the signal*

Overfitting (high variance)

*Fits noise as if
it were signal*

REGULARIZATION

L2 Regularization & Weight Decay

- The most common form is L2 regularization (L2 shrinkage or weight decay).
- λ is the regularization parameter; increasing it strengthens regularization.
- We add the L2 term to the cost function, which shrinks the weights — equivalent to placing a zero-mean Gaussian prior over them.
- Since the derivative of w^2 is $2w$, gradient descent naturally decays each weight in proportion to its size.

$$\frac{\lambda}{2} \|\mathbf{w}\|^2 = \frac{\lambda}{2} \sum_{j=1}^m w_j^2$$

$$J(\mathbf{w}) = \sum_{i=1}^n [-y^{(i)} \log \phi(z^{(i)}) - (1 - y^{(i)}) \log(1 - \phi(z^{(i)}))] + \frac{\lambda}{2} \|\mathbf{w}\|^2$$
$$w_i = w_i - \eta \frac{\partial J}{\partial w_i} - \eta \lambda w_i$$

The Regularization Parameter C

- We control the fit-vs-simplicity trade-off through λ ; scikit-learn (borrowing SVM terminology) instead exposes the inverse, $C = 1/\lambda$.
- Decreasing C increases regularization strength — weights shrink toward zero.
- The regularized cost function below is the SVM-style rewrite used throughout scikit-learn's LogisticRegression and LinearSVC.

$$C = \frac{1}{\lambda}$$

Large C

weak regularization, large error penalty — risk of overfitting

Small C

strong regularization, tolerant of misclassification — risk of underfitting

SECTION 04

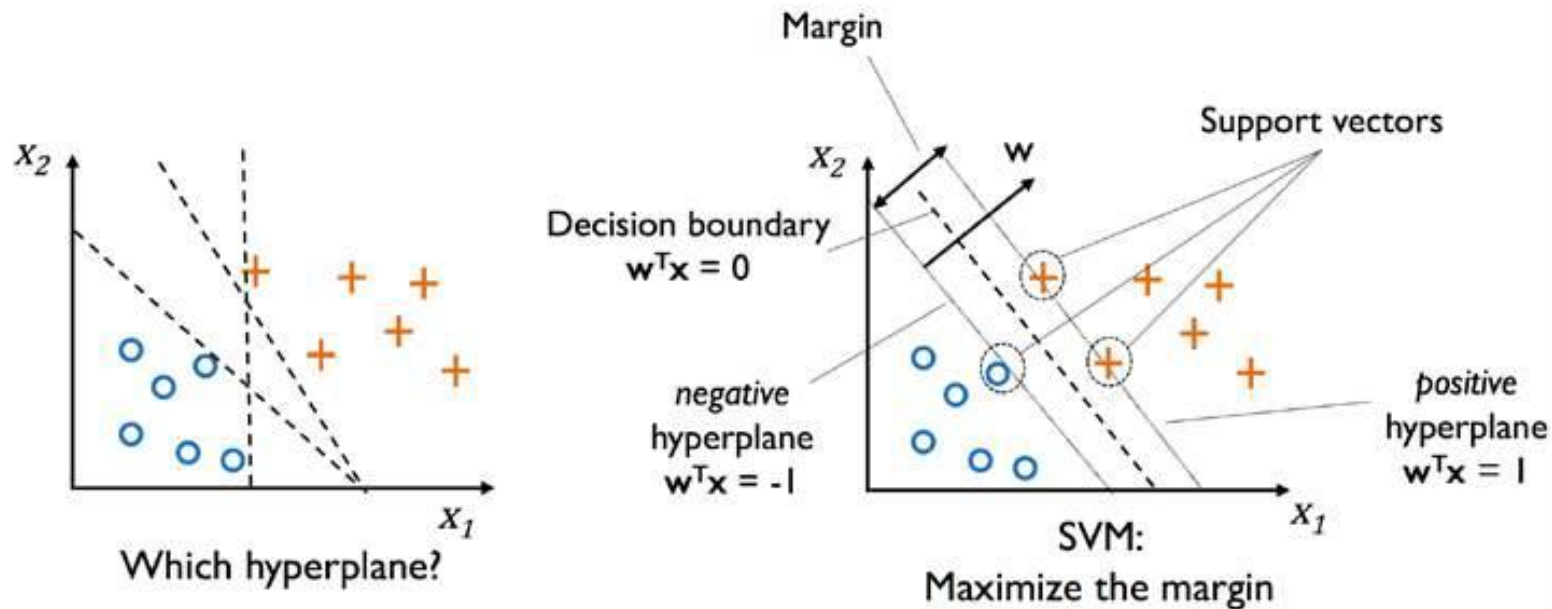
Support Vector Machines

Maximizing the margin between classes for better generalization

MAXIMUM MARGIN

Maximum-Margin Classification

- SVM's optimization objective is to maximize the margin.
- The margin is the distance between the separating hyperplane and the closest training samples — the support vectors.
- Intuitively, a larger margin means lower generalization error.
- Models trained with a small margin are prone to overfitting.



The Mathematics of the Margin

- The positive and negative hyperplanes run parallel to the decision boundary, offset by ± 1 in the net-input scale.
- The distance between these two planes — the margin — works out to $2 / \|w\|$.
- $\|w\|$ is the Euclidean length (norm) of the weight vector.

$$w_0 + w^T x_{pos} = 1 \quad w_0 + w^T x_{neg} = -1$$

$$\Rightarrow w^T (x_{pos} - x_{neg}) = 2$$

$$\frac{w^T (x_{pos} - x_{neg})}{\|w\|} = \frac{2}{\|w\|} \quad \|w\| = \sqrt{\sum_{j=1}^m w_j^2}$$

The Constrained Optimization Problem

- Objective function of SVM becomes maximization of: $\text{margin} = \frac{2}{\|\mathbf{w}\|}$
- Maximizing the margin ($2/\|\mathbf{w}\|$) is equivalent to minimizing $\frac{1}{2}\|\mathbf{w}\|^2$ — a much friendlier quadratic objective,

$$\text{minimize } \frac{1}{2} \|\mathbf{w}\|^2$$

- with the constraint that every training sample must fall on the correct side of its class's hyperplane.

$$w_0 + \mathbf{w}^T \mathbf{x}^{(i)} \geq 1 \quad \text{if } y^{(i)} = 1 \quad w_0 + \mathbf{w}^T \mathbf{x}^{(i)} \leq -1 \quad \text{if } y^{(i)} = -1$$

- Both constraints can be written compactly as a single inequality that must hold for every sample i .

$$y^{(i)} (w_0 + \mathbf{w}^T \mathbf{x}^{(i)}) \geq 1 \quad \forall i$$

MAXIMUM MARGIN

The SVM Solution

- Solving the constrained optimization problem (via its Lagrangian dual) yields the classifier and its weight vector below.
- The weight vector is a weighted sum over training points — but the Lagrange multipliers α_i are zero for every point that isn't a support vector.
- In practice, the decision boundary is defined only by the support vectors, not the whole training set.

$$f(\mathbf{x}) = \text{sgn}(\mathbf{w}^T \mathbf{x} + w_0)$$

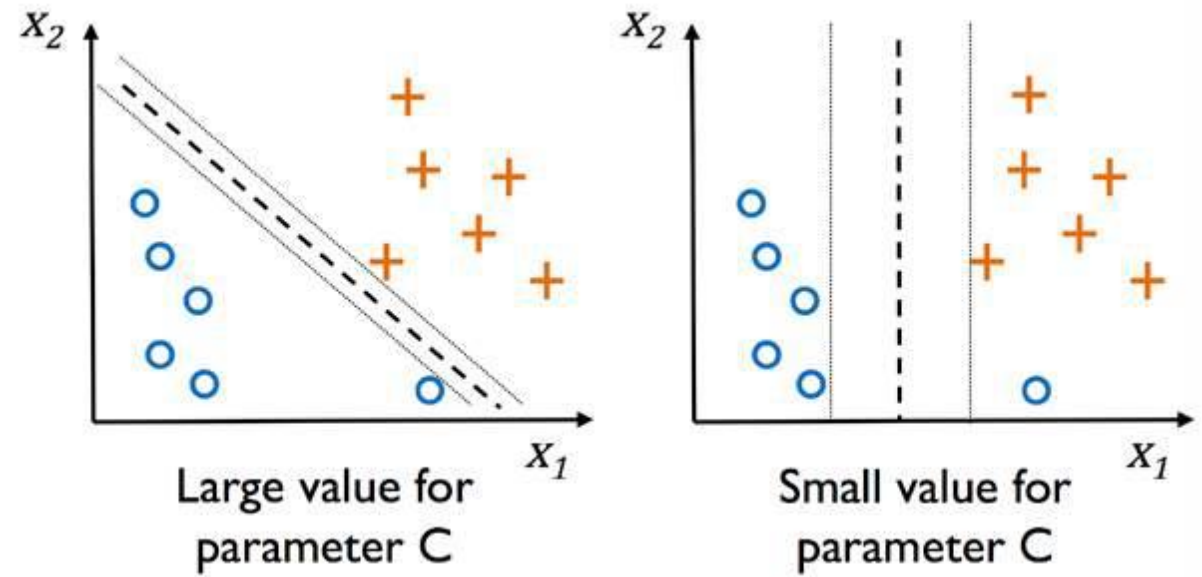
classifier

$$\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$$

weights, from support vectors only (nonzero α_i)

Slack Variables: the Soft-Margin SVM

- Real data is rarely perfectly separable — the hard-margin constraints need relaxing.
- A slack variable ξ_i is introduced per sample, allowing it to fall inside (or across) the margin.
- Each violation is penalized, but convergence is now guaranteed even with misclassifications.



$$\mathbf{w}^T \mathbf{x}^{(i)} \geq 1 - \xi^{(i)} \quad (y^{(i)} = 1) \quad \mathbf{w}^T \mathbf{x}^{(i)} \leq -1 + \xi^{(i)} \quad (y^{(i)} = -1)$$

Regularization in SVMs: the Role of C

- Minimizing $\frac{1}{2}\|\mathbf{w}\|^2 + C \cdot \sum \xi_i$ balances margin width against total slack.
- Large C: large penalties for misclassification — narrower margin, closer fit.
- Small C: more tolerant of misclassification — wider margin, more regularization.
- C therefore plays exactly the same role here as it does in regularized logistic regression.

$$\frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi^{(i)}$$

Large C
narrow margin

Small C
wide margin

SECTION 05

Kernel Methods

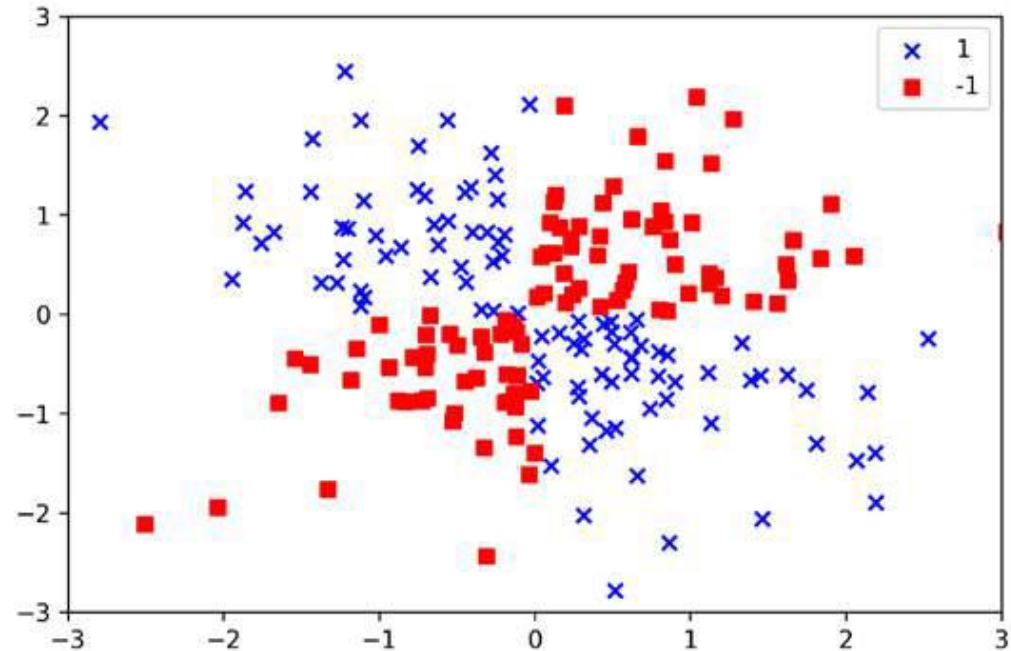
Solving problems that no straight line can separate

THE NON-LINEAR PROBLEM

When a Straight Line Isn't Enough

- The classic Exclusive-OR (XOR) problem is not linearly separable: no single straight line divides the two classes.

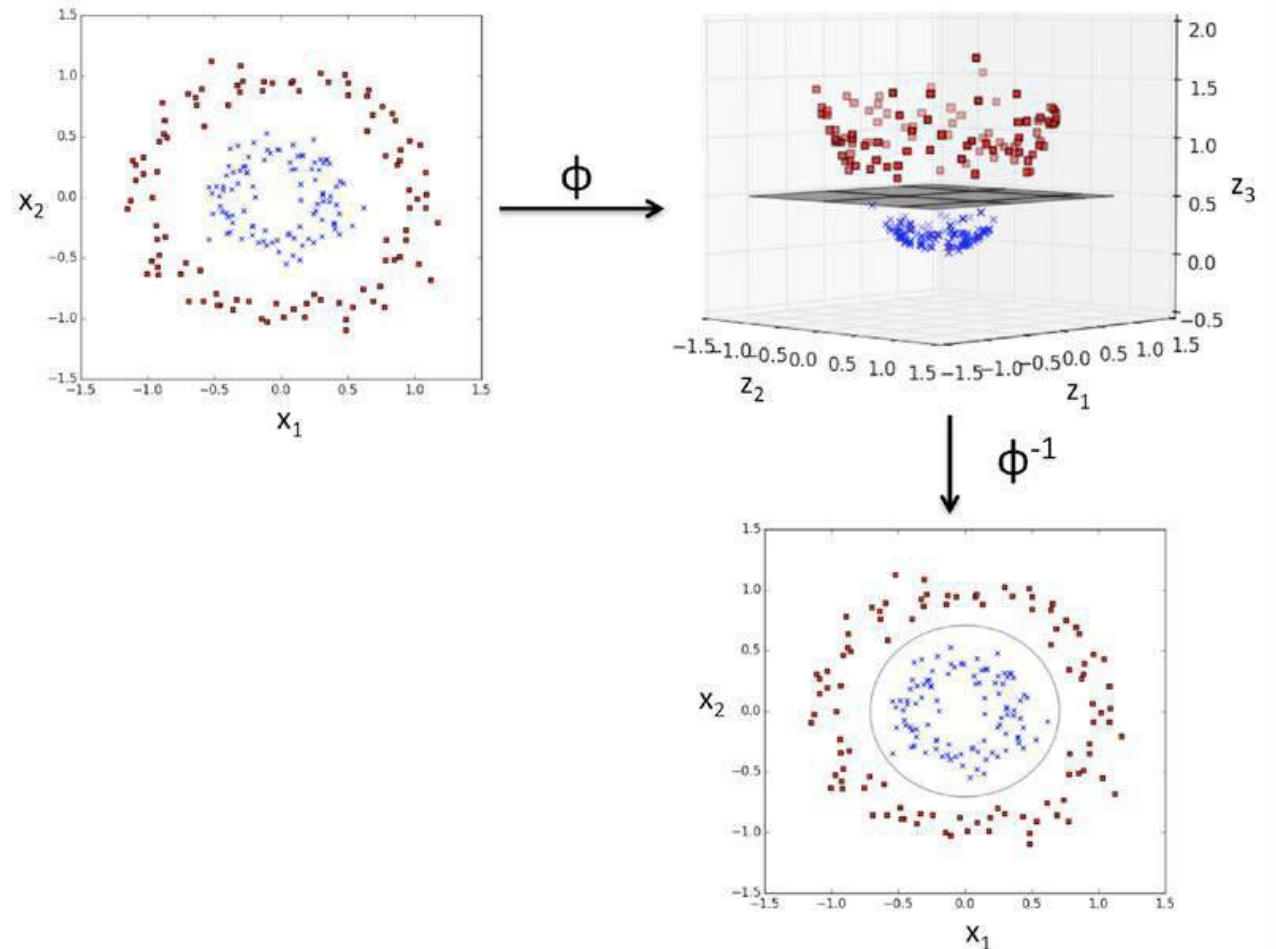
X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0



Kernel methods for linearly inseparable data

- Kernel methods build non-linear combinations of the original features.
- By projecting onto a higher-dimensional space, classes that overlap in 2D become linearly separable.

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$$



The Kernel Trick: General Blueprint

- 1 Transform training data into a higher-dimensional space via a mapping function $\phi(\cdot)$.
- 2 Train a linear SVM to classify the data in that new feature space.
- 3 Apply the same mapping $\phi(\cdot)$ to transform new, unseen data.
- 4 Classify unseen data using the linear SVM model.

Problem: constructing ϕ explicitly is computationally expensive for high dimensions.

Avoiding the Explicit Mapping

- The decision boundary only ever needs the dot product of two feature vectors — never the vectors' individual coordinates in the new space.
- So, we define a kernel function that computes $\phi(\mathbf{x})^T \phi(\mathbf{z})$ directly, without ever materializing ϕ .
- Worked example: squaring the dot product $\langle \mathbf{x} \cdot \mathbf{z} \rangle$ implicitly computes the dot product of a 3-D feature mapping — with none of the expansion cost.

$$k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(j)})$$

kernel function — replaces the dot product

$$K(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x} \cdot \mathbf{z} \rangle^2 = \langle \phi(\mathbf{x}) \cdot \phi(\mathbf{z}) \rangle$$

worked example (polynomial kernel)

The RBF (Gaussian) Kernel

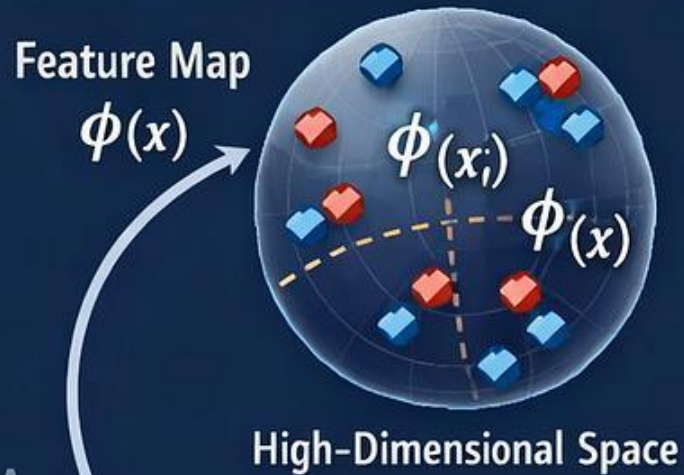
- The Radial Basis Function kernel is the most widely used kernel in practice.
- $\gamma = 1/2\sigma^2$ is a free parameter, tuned like any other hyperparameter.
- A kernel can be read as a similarity function between two samples — the minus sign inverts distance into similarity (0 = dissimilar, 1 = identical).
- Varying γ on the XOR data traces out very different decision boundaries

$$k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2)$$

$$\gamma = \frac{1}{2\sigma^2}$$

The Kernel Trick

Feature Space



$$\phi(x_i) \cdot \phi(x_i)$$

Inner Product

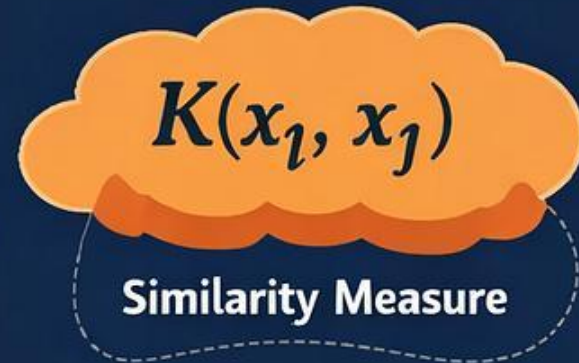
Kernel Substitution

Replace with $\Rightarrow$

$$\phi(x_i) \cdot \phi(x_j) \leftrightarrow K(x_i, x_j)$$

No need to compute $\phi(x)$

Kernel Function



- Linear Kernel
- Polynomial Kernel
- RBF Kernel

Algorithm becomes:

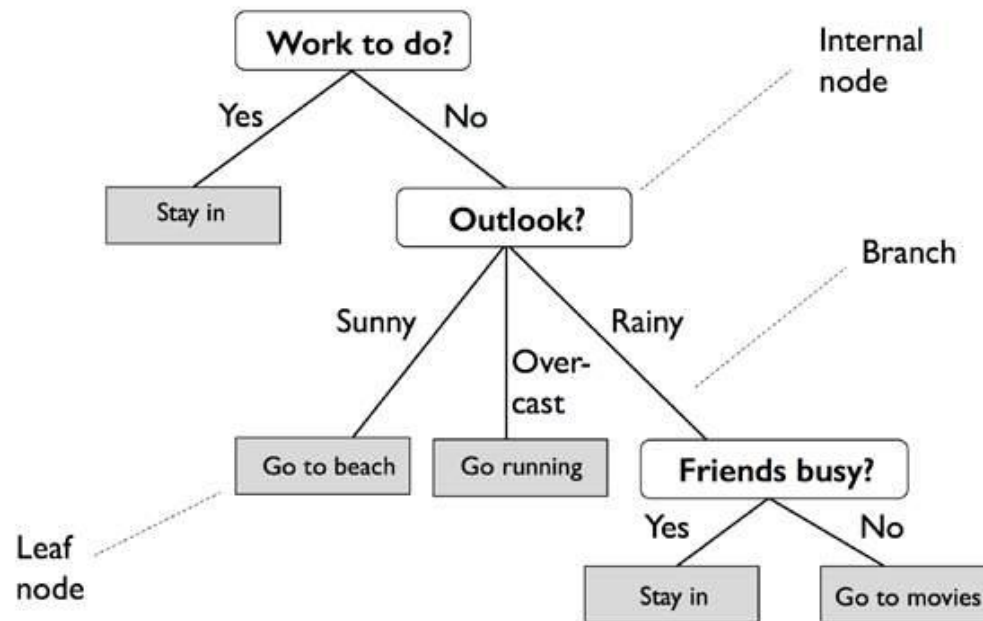
$$\sum \alpha_i K(x_i, x_j)$$

Decision Trees & Random Forests

Splitting on the most informative features, measuring impurity,
and combining trees into a robust ensemble

Breaking a decision into questions

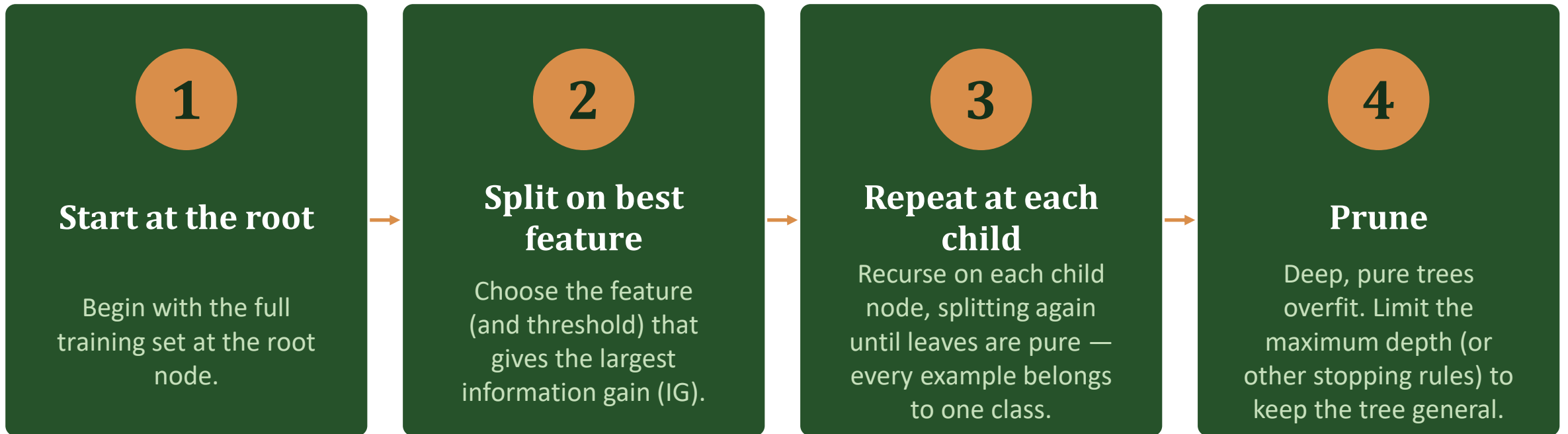
- Decision trees are attractive when interpretability matters: the model asks a series of yes/no questions and arrives at a decision.



Once fit to real-valued features (like the Iris dataset), the same idea applies: instead of “Outlook?”, the tree asks a threshold question, e.g. “Is sepal width ≥ 2.8 cm?”

THE ALGORITHM

Growing a tree, step by step



A very deep tree with fully pure leaves can easily overfit — that's why pruning (e.g. capping `max_depth`) matters in practice.

THE OBJECTIVE

Maximizing information gain

To decide where to split, we define an objective function: maximize the information gain (IG) at each split.

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m (N_j / N_p) I(D_j)$$

f the feature used to perform the split

I the impurity measure (entropy, Gini, or classification error)

D_p, D_j the dataset of the parent node, and of the j-th child node

N_p, N_j the number of training examples at the parent node, and at child j

For simplicity — and to shrink the search space — most libraries (including scikit-learn) grow strictly binary trees: every parent node splits into exactly two children, D_{left} and D_{right} .

SPLITTING CRITERIA

Three ways to measure impurity

Entropy

$$I_H(t) = -\sum p(i|t) \log_2 p(i|t)$$

0 when a node is pure;
maximal (1, for 2 classes)
when classes are perfectly
mixed. Aims to maximize
mutual information.

Gini impurity

$$I_G(t) = 1 - \sum p(i|t)^2$$

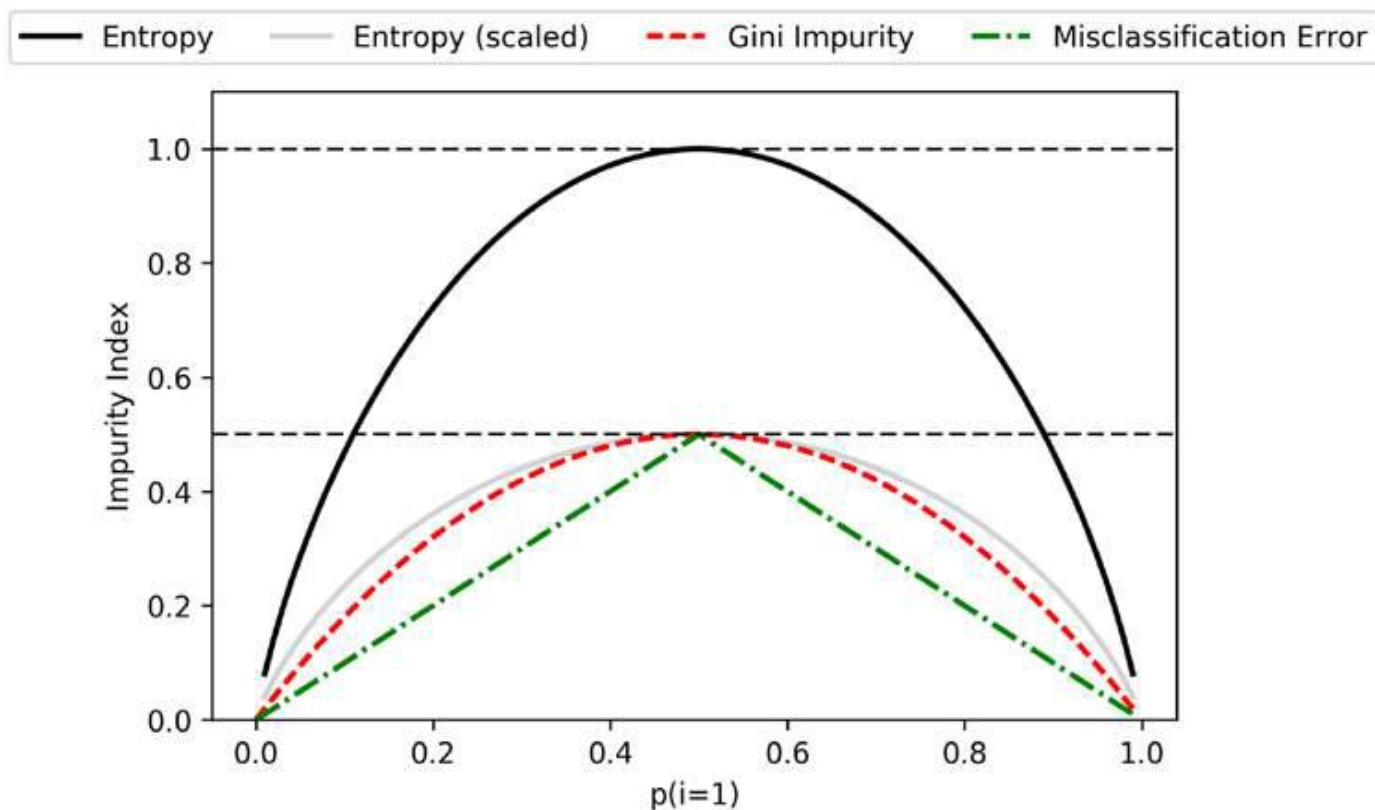
Minimizes the probability
of misclassifying a
randomly picked example.
Also maximal for a perfectly
mixed node.

Classification error

$$I_E(t) = 1 - \max\{ p(i|t) \}$$

Useful for pruning, but not
recommended for growing
— it's less sensitive to
changes in class
probabilities.

Comparing the three criteria visually



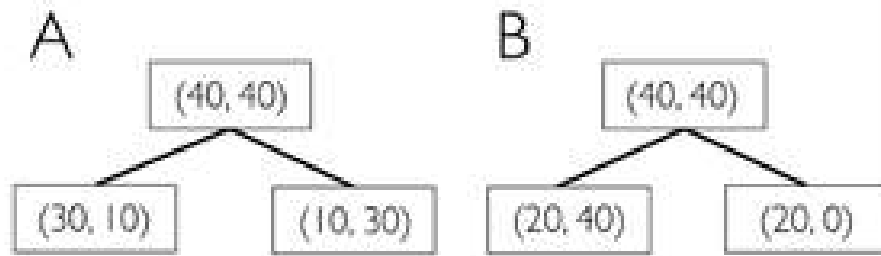
Reading the curves

- All three measures peak when classes are perfectly mixed ($p = 0.5$) and hit 0 at the pure extremes.
- Gini sits between entropy and misclassification error — an intermediate measure.
- In practice, Gini and entropy usually produce very similar trees.
- Misclassification error is far less sensitive mid-range, which is why it's a poor choice for growing a tree.

A WORKED COMPARISON

Same IG, different quality of split

Parent node D_p : 40 examples of class 1 and 40 of class 2. Two possible splits:



Both children are still fairly mixed.

The right child is already pure — clearly the better split.

Classification error gives both scenarios the same IG (0.25) — it can't tell that B is the better split. Gini impurity and entropy, however, correctly favor scenario B, because they're more sensitive to how pure each child actually is.

information gain using the classification error

$$I_E(D_p) = 1 - 0.5 = 0.5$$

$$A: I_E(D_{left}) = 1 - \frac{3}{4} = 0.25$$

$$A: I_E(D_{Right}) = 1 - \frac{3}{4} = 0.25$$

$$A: IG_E = 0.5 - \frac{4}{8}0.25 - \frac{4}{8}0.25 = 0.25$$

$$B: I_E(D_{left}) = 1 - \frac{4}{6} = \frac{1}{3}$$

$$B: I_E(D_{Right}) = 1 - 1 = 0$$

$$B: IG_E = 0.5 - \frac{6}{8} \times \frac{1}{3} - 0 = 0.25$$

Building a decision tree with scikit-learn

Gini impurity would favor the split in scenario B ($I_G = 0.1\bar{6}$) over scenario A ($I_G = 0.125$), which is indeed purer

$$I_G(D_p) = 1 - (0.5^2 + 0.5^2) = 0.5$$

$$A: I_G(D_{left}) = 1 - \left(\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right) = \frac{3}{8} = 0.375$$

$$A: I_G(D_{right}) = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right) = \frac{3}{8} = 0.375$$

$$A: IG_G = 0.5 - \frac{4}{8} 0.375 - \frac{4}{8} 0.375 = 0.125$$

$$B: I_G(D_{left}) = 1 - \left(\left(\frac{2}{6} \right)^2 + \left(\frac{4}{6} \right)^2 \right) = \frac{4}{9} = 0.\bar{4}$$

$$B: I_G(D_{right}) = 1 - (1^2 + 0^2) = 0$$

$$B: IG_G = 0.5 - \frac{6}{8} 0.\bar{4} - 0 = 0.1\bar{6}$$

Entropy criterion would also favor scenario B ($I_H = 0.31$) over scenario A ($I_H = 0.19$):

$$I_H(D_p) = -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) = 1$$

$$A: I_H(D_{left}) = - \left(\frac{3}{4} \log_2 \left(\frac{3}{4} \right) + \frac{1}{4} \log_2 \left(\frac{1}{4} \right) \right) = 0.81$$

$$A: I_H(D_{right}) = - \left(\frac{1}{4} \log_2 \left(\frac{1}{4} \right) + \frac{3}{4} \log_2 \left(\frac{3}{4} \right) \right) = 0.81$$

$$A: IG_H = 1 - \frac{4}{8} 0.81 - \frac{4}{8} 0.81 = 0.19$$

$$B: I_H(D_{left}) = - \left(\frac{2}{6} \log_2 \left(\frac{2}{6} \right) + \frac{4}{6} \log_2 \left(\frac{4}{6} \right) \right) = 0.92$$

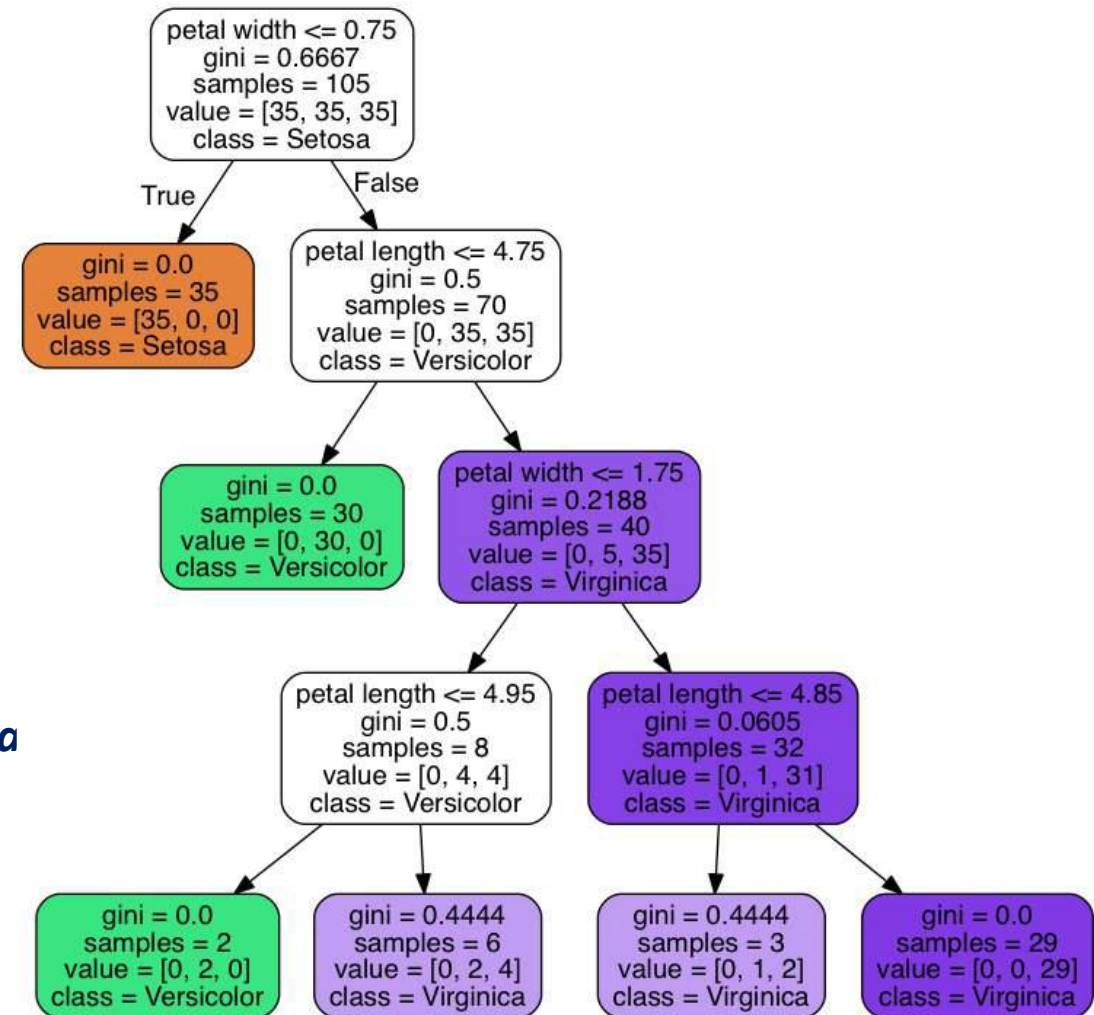
$$B: I_H(D_{right}) = 0$$

$$B: IG_H = 1 - \frac{6}{8} 0.92 - 0 = 0.31$$

Visualize the decision tree model after training

- scikit-learn (and Graphviz, for nicer output) can render the trained tree's structure directly — every node shows its splitting rule, impurity, sample count, and class distribution.

The root splits on petal width first; the left child is already pure (all Setosa). Deeper splits on petal length continue separating Versicolor from Virginica



ENSEMBLES

Random forests: many trees, one vote

A random forest averages many (deep) decision trees, each of which individually suffers from high variance, to build a more robust, better-generalizing model.



Sampling with replacement:

In random sampling with replacement, we always return the selected number to the urn so that the probability of drawing a particular number at each turn does not change; we can draw the same number more than once. In other words, in sampling with replacement, the samples (numbers) are independent and have a covariance of zero.

Sampling without replacement:

In sampling without replacement, we do not put the number back into the urn after each turn. Consequently, the probability of drawing a particular number from the set of remaining numbers in the next round depends on the previous round.

Hyperparameters and key takeaways

Parameters that matter

k — number of trees

More trees generally improve performance, at higher computational cost.

n — bootstrap sample size

Usually set equal to the training set size; smaller n increases tree diversity (and variance).

d — features per split

A common default is $\sqrt{m}$, where m is the total number of features.

Takeaways

- Single decision trees are interpretable but prone to overfitting without pruning.
- Gini and entropy work well for growing trees; classification error is better reserved for pruning.
- Random forests trade interpretability for robustness — and rarely need pruning themselves.
- scikit-learn's RandomForestClassifier already implements the full algorithm — no need to build it from scratch.

SECTION 06

K-Nearest Neighbors

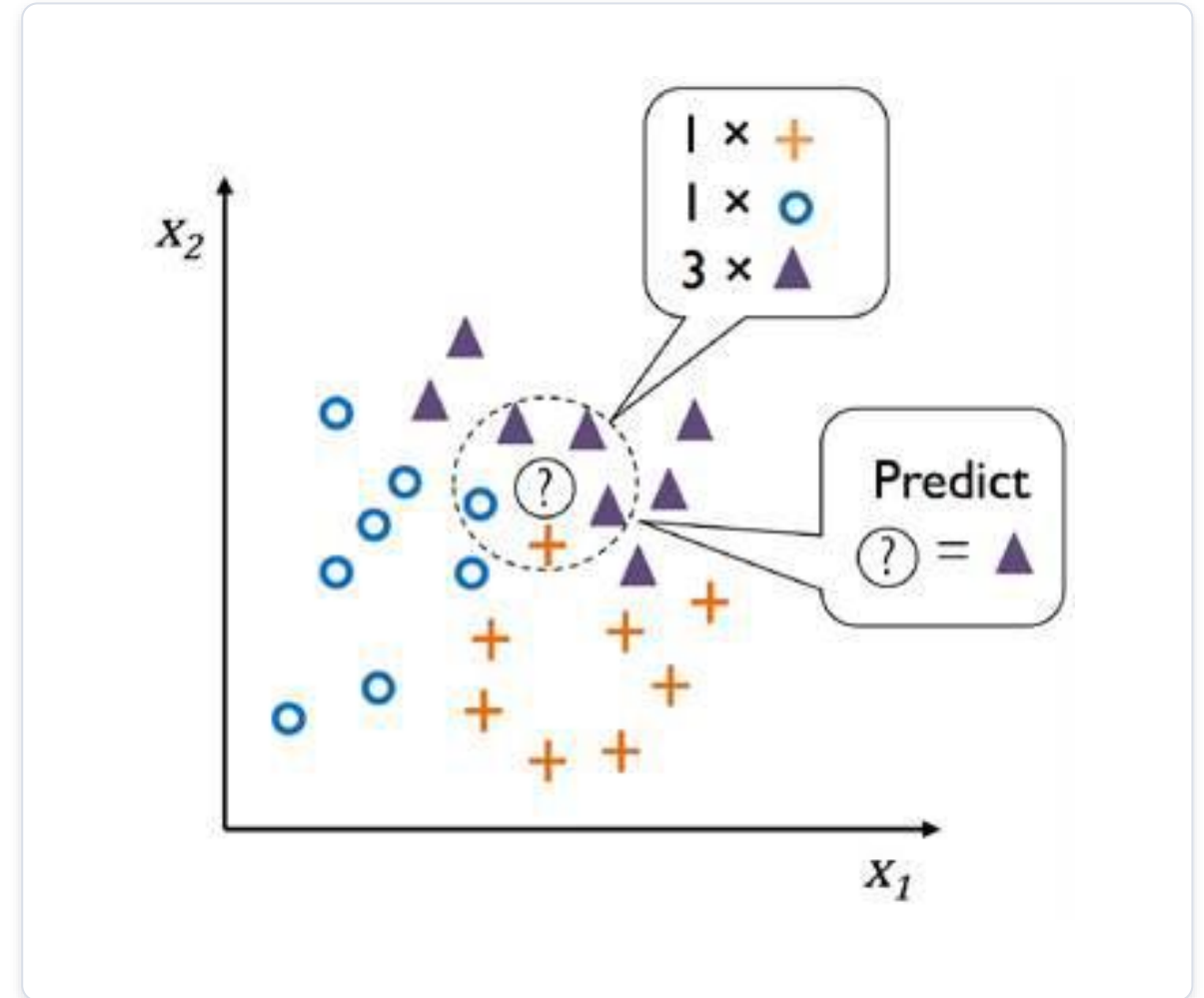
A simple, non-parametric, instance-based classifier

INSTANCE-BASED LEARNING

K-Nearest Neighbors: the Algorithm

- KNN is a non-parametric model — it learns no fixed set of parameters from the training data.
- Unlike parametric models, KNN needs the entire training set at prediction time — it's an instance-based learner.

- 1 Choose k and a distance metric
- 2 Find the k nearest neighbors of the query point
- 3 Assign the class label by majority vote



The curse of dimensionality

The curse of dimensionality describes the phenomenon where the feature space becomes increasingly sparse for an increasing number of dimensions of a fixed-size training dataset. KNN is very susceptible to overfitting due to the curse of dimensionality. We can think of even the closest neighbors as being too far away in a high dimensional space to give a good estimate.

Trade-offs & Distance Metrics

- Advantage: the classifier immediately adapts as new training examples arrive — no retraining step.
- Cost: computational complexity grows linearly with the number of samples (curse of dimensionality); efficient structures like KD-trees help at scale.

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt[p]{\sum_k |x_k^{(i)} - x_k^{(j)}|^p}$$

p = 2

Euclidean distance

p = 1

Manhattan distance

SUMMARY

Choosing the Right Classifier

Logistic Regression

Probability estimates, fast, interpretable

Assumes a fairly linear decision boundary

SVM (linear)

Robust margins, works well in high dimensions

Less naturally probabilistic

SVM (kernel / RBF)

Handles non-linear boundaries well

Slower on very large datasets, needs tuning

K-Nearest Neighbors

Simple, no training phase, adapts instantly

Slow at prediction time, sensitive to scale

Remember: no free lunch — always match the model to the problem.